基于Nordic AI的智能戒指手势识别

版本

版本号	变更内容	作者	时间
1.0.0	- 初始版本；	moke	2026.3.25

描述

该文档为使用NRF54L15DK以及进行AI手势识别，包含采集数据集的上位机，训练模型，使用模型的全流程
所需上位机地址https://github.com/chunnuanhuakai666/nrf54L15AI_Gesture_Recognition_Host (https://github.com/chunnuanhuakai666/nrf54L15AI_Gesture_Recognition_Host)

基于Nordic AI的智能戒指手势识别

1. 运行环境

1. 软件

Edge AI Add-on v2.0.0（目前为实验性版本）基于 nRF Connect SDK v3.3.0-preview2

在 VS Code 中安装 Edge AI Add-on

1. **点击 Activity Bar 中的 nRF Connect 图标 ，打开扩展**
2. **在 Welcome View 中点击 Create a new application**
3. **选择 Browse nRF Connect SDK Add-on Index**
4. **在列表中选择 nRF Edge AI Add-on**
5. **选择要安装的 Add-on 版本**

安装如果遇到128网络问题报错，可在下载路径中运行

```
west updata
```

2. 硬件

1. **NRF54L15DK**
2. **BMI270（3轴加速度计 + 3轴陀螺仪）**
3. **连接方式（SCL -> P1.11， SDA -> P1.12）**

2.数据采集

1. 驱动BMI270（3轴加速度计 + 3轴陀螺仪）

例程位置zephyr\samples\sensor\bmi270

参数	加速度计设置	陀螺仪设置
量程 (Full Scale)	±2 G	±500 dps (度/秒)
采样率 (Sampling Frequency)	100 Hz	100 Hz
采样周期	10 ms	10 ms
过采样 (Oversampling)	Normal mode (×1)	Normal mode (×1)

2. 修改app.overlay

替换为以下内容

```
/*
 * Copyright (c) 2021 Bosch Sensortec GmbH
 *
 * SPDX-License-Identifier: Apache-2.0
 */

&spi21 { status = "disabled"; };
&uart21 { status = "disabled"; };

&i2c21 {
    status = "okay";
    compatible = "nordic,nrf-twim";
    pinctrl-0 = <&i2c21_default>;
    pinctrl-1 = <&i2c21_sleep>;
    pinctrl-names = "default", "sleep";
    clock-frequency = <I2C_BITRATE_FAST>;
    zephyr,concat-buf-size = <257>;

    bmi270: bmi270@68 {
        compatible = "bosch,bmi270";
        reg = <0x68>;
```

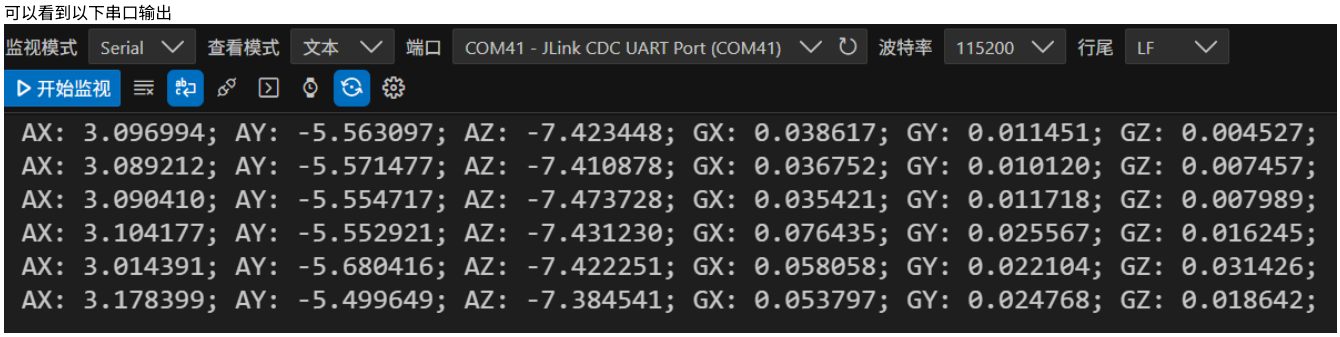
3. 修改main.c

修复负号显示问题

```
/*
 * @Author: kiki kiki@xbeacon.net
 * @Date: 2026-03-19 15:33:56
 * @LastEditors: kiki kiki@xbeacon.net
 * @LastEditTime: 2026-03-23 12:47:25
 * @FilePath: \bmi270\src\main.c
 * @Description: 这是默认设置,请设置`customMade`, 打开koroFileHeader查看配置 进行设置: https://github.com/OBKoro1/koro1FileHeader/wiki/%E9%85%8D%E7%BD%AE
 */
/*
 * Copyright (c) 2021 Bosch Sensortec GmbH
 *
 * SPDX-License-Identifier: Apache-2.0
 */

#include <zephyr/kernel.h>
#include <zephyr/device.h>
#include <zephyr/drivers/sensor.h>
#include <stdio.h>

int main(void)
{
```

编译下载

可以看到以下串口输出



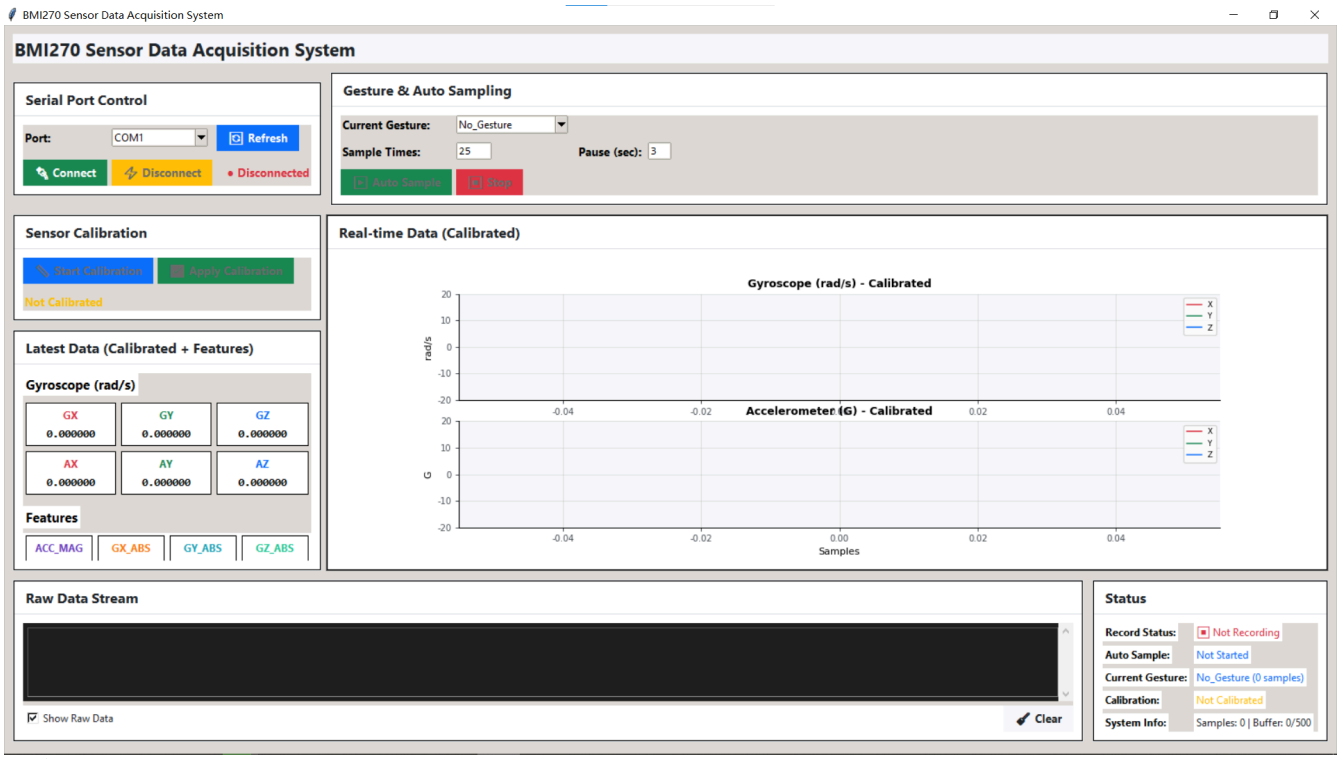
2.2 python上位机

1. 环境

Python 3.7+（推荐 3.8-3.12）

库	用途	安装命令
pyserial	串口通信	`pip install pyserial`
numpy	数值计算、校准数据处理	`pip install numpy`
matplotlib	实时数据绘图	`pip install matplotlib`
tkinter	GUI界面（Python标准库）	通常已内置
pandas	数据处理	

2. 运行app.py



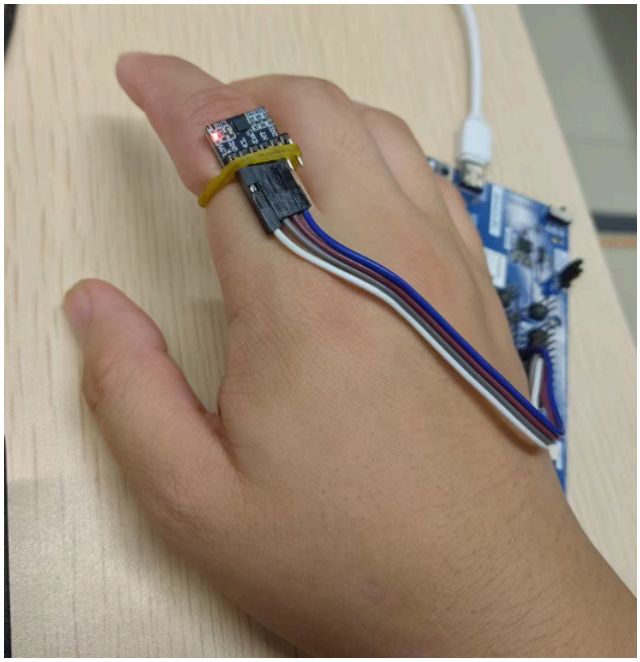
使用方法：

- 串口连接：左侧偏上选择COM号连接开发板
- 传感器校准：连接下方
- 数据采集：连接右侧，点击后选择csv文件的保存位置，每次采集1S，可选总采集时间和单次采集后的等待时间（手势数据建议最低20次以上）

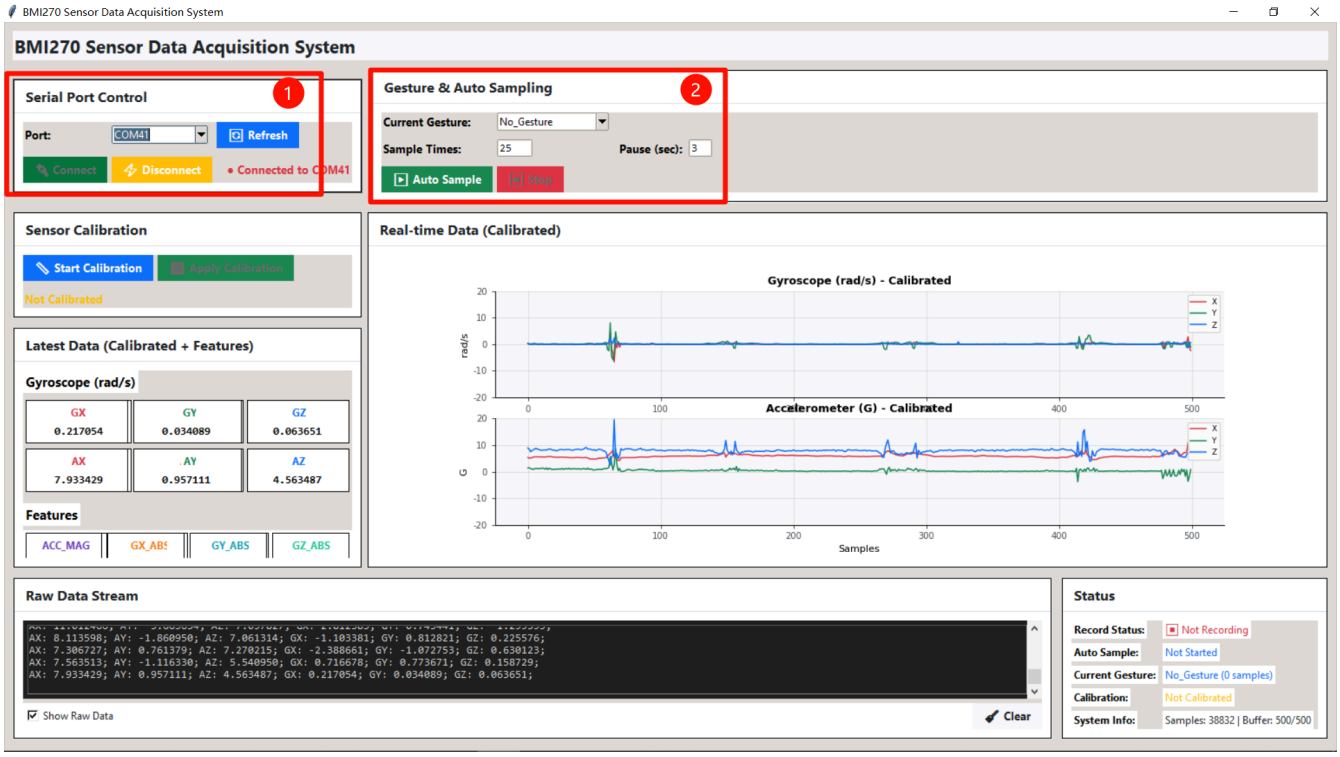
注意：点击开始采集后会弹出窗口倒计时，倒计时结束后显示GO，然后窗口消失，采集的这1s数据是从窗口消失开始

3. 采集流程

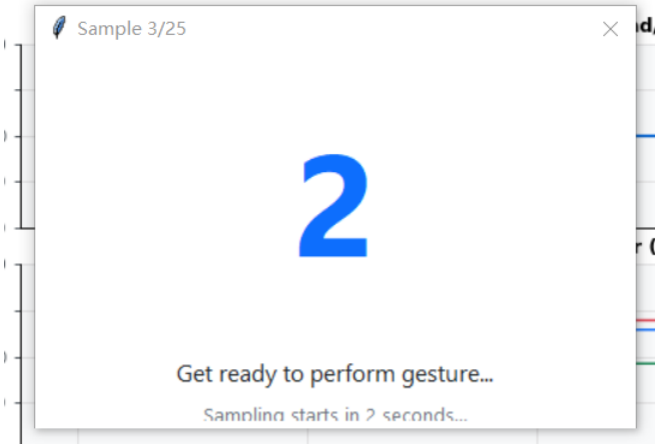
1. 将BMI270固定在食指



2. 然后上位机选择COM连接54L15，点击开始采集（假设采集时间为25s，则弹窗25次每次采集1s）



3. 等待倒计时结束且窗口消失



4. 窗口消失后在1S内做完手势，然后回到无动作状态，等待下一次弹窗

这里训练了两个手势，点击和右滑

3 数据预处理

1. 将采集到的原始数据文件（csv）放在data文件夹中

2. 运行data_processing.py，得到可以直接训练的文件merged_gesture_data.csv，同时终端也会显示手势类别转换后数字标签

```
==================================================
Class Mapping (Gesture -> Numeric Label):
==================================================
  0: Single_Click (Total: 1250 samples)
  1: Swipe_Right (Total: 1250 samples)
```

data_processing.py：

- 把数据转换为可直接放入eage ai训练的格式
- 简化数据格式，加速训练
- 数据集中每次动作的窗口转换为50行
- 合并所有手势文件

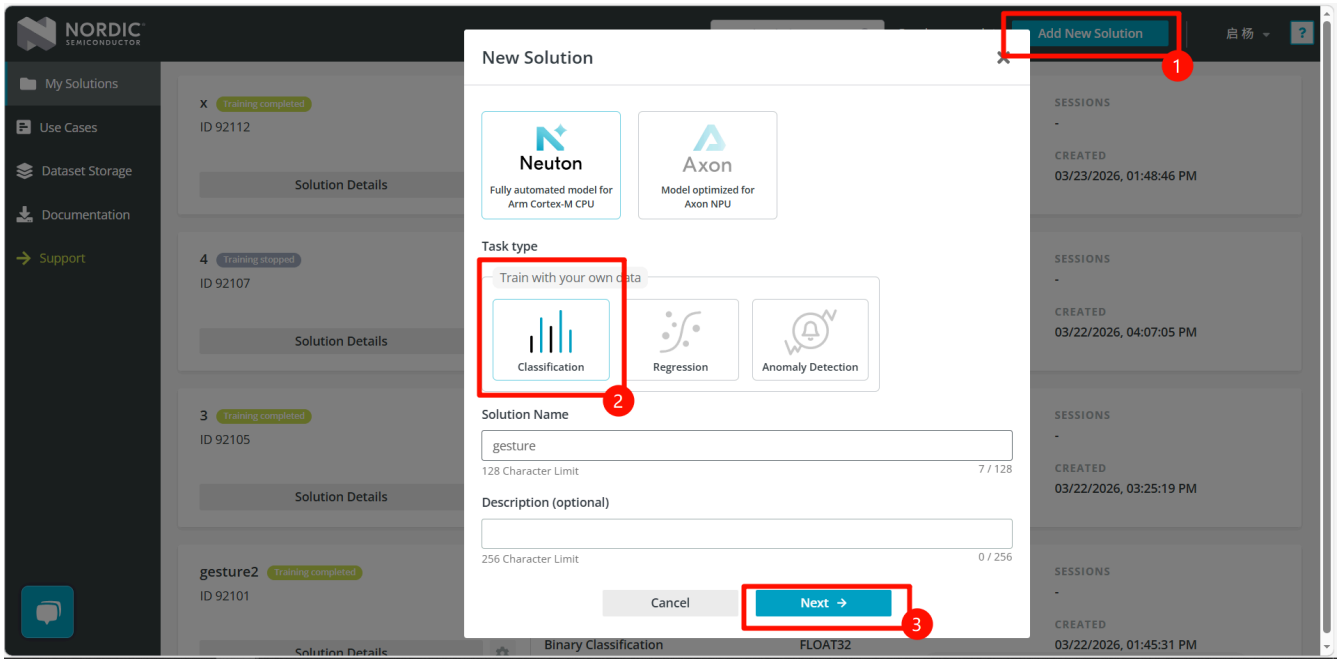
4 Edge AI Lab 模型训练

1. 创建 Solution

平台链接：[Nordic Edge AI Lab (https://app.lab.nordicsemi.com/)](https://app.lab.nordicsemi.com/)

在平台上新建一个 Solution，作为管理训练参数、结果和分析的工作空间。

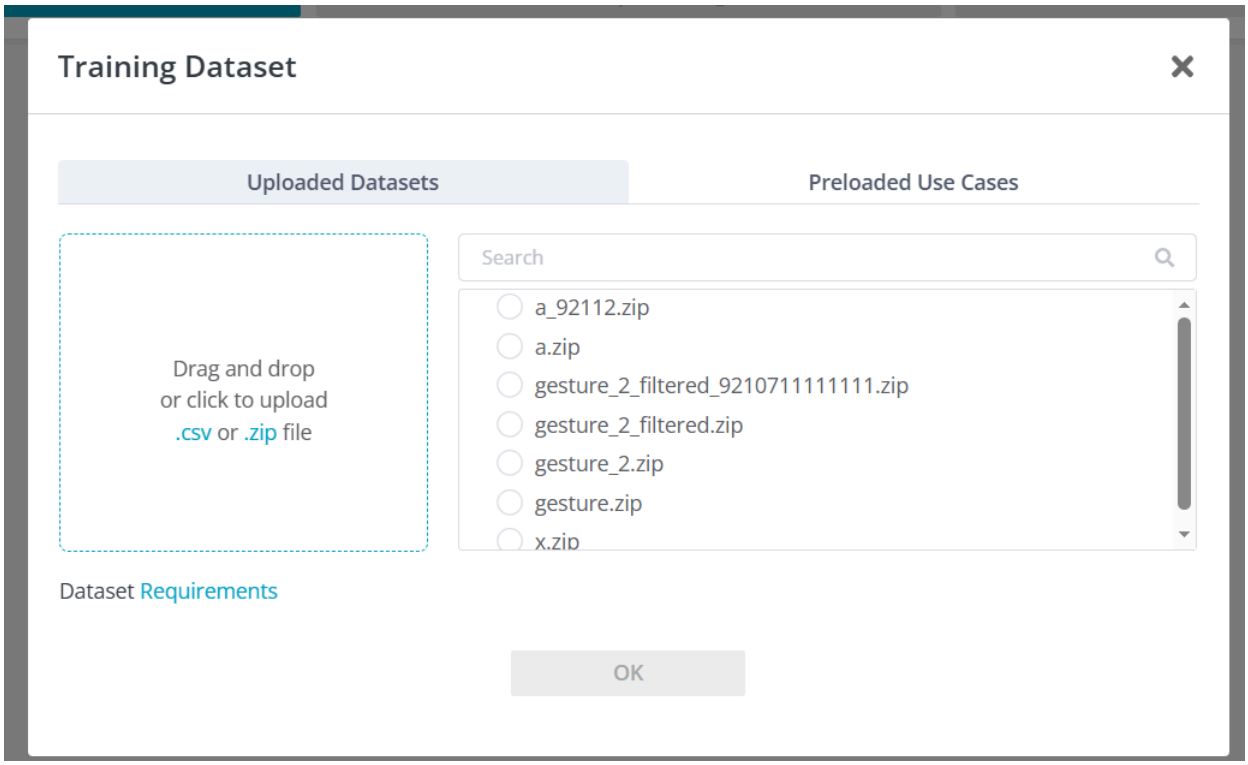
以手势分类为例



2. 上传数据集

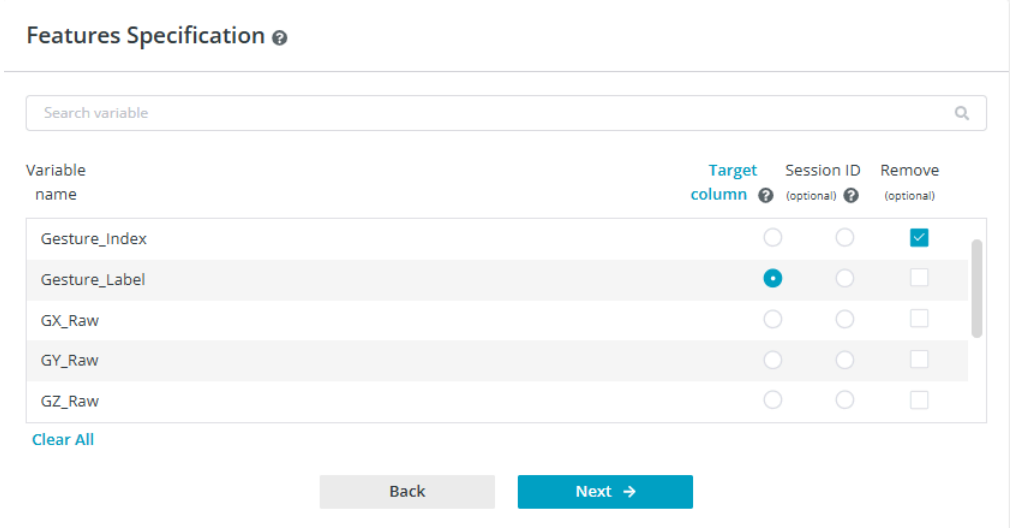
以 **CSV 格式**上传已标注的数据集，并定义目标变量（Target Column）。数据集要求：

- **UTF-8 编码**
- **第一行为列名（header）**
- **包含特征列和标签列** [Overview of Edge AI Lab (https://docs.nordicsemi.com/bundle/edge-ai-lab/page/doc/overview_of_nordic_edge_ai_lab.html#workflow)]
  选择刚才预处理后的数据上传

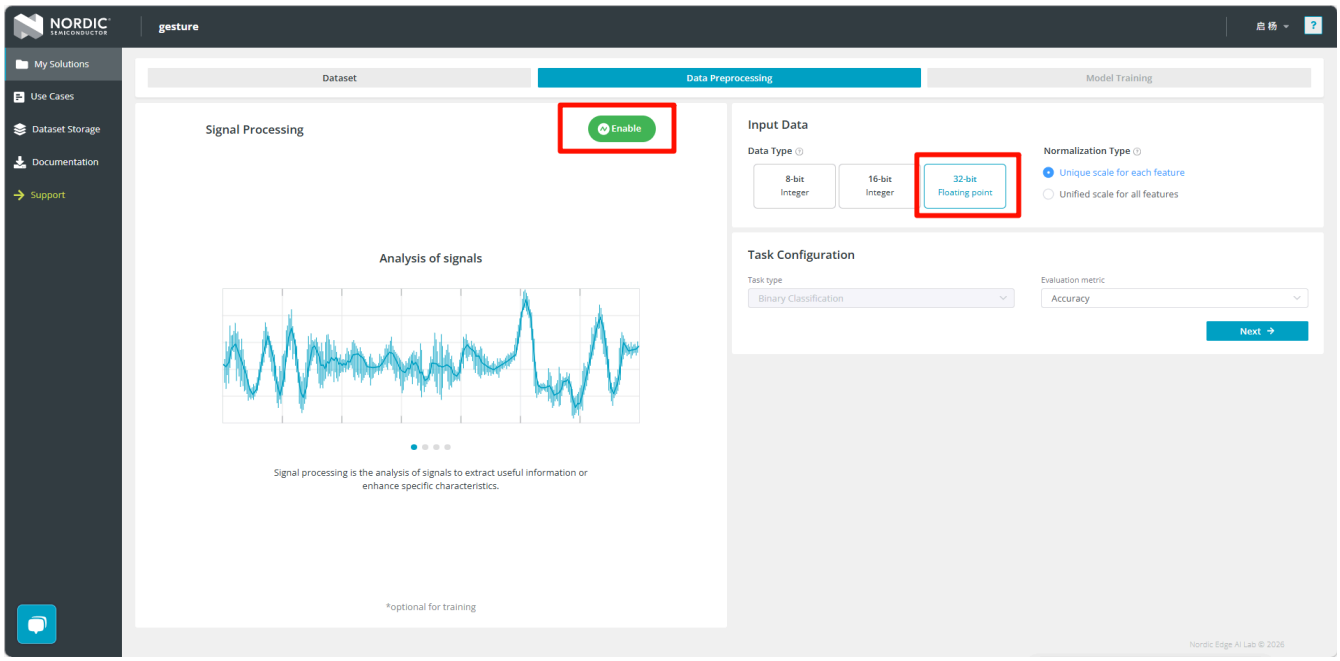


3. 配置模型参数（Model Settings）

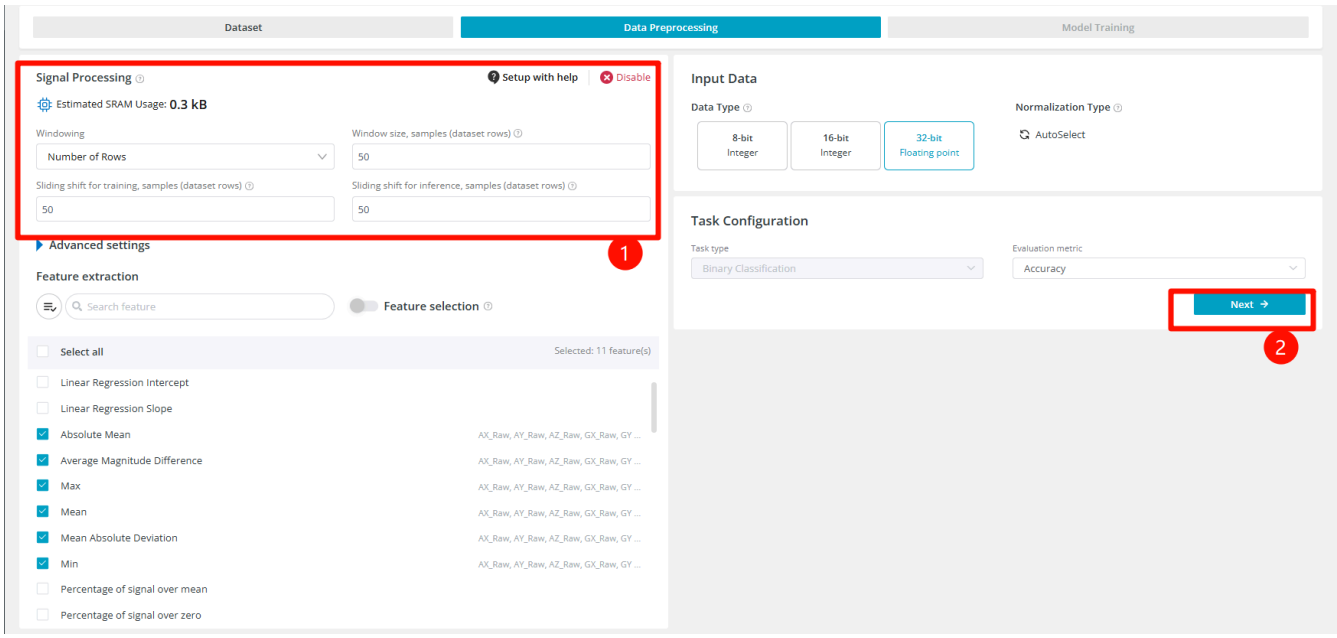
选择需要识别的列为目标列（其余为特征）,并删除序号列



开启信号处理，数据类型为32-bit

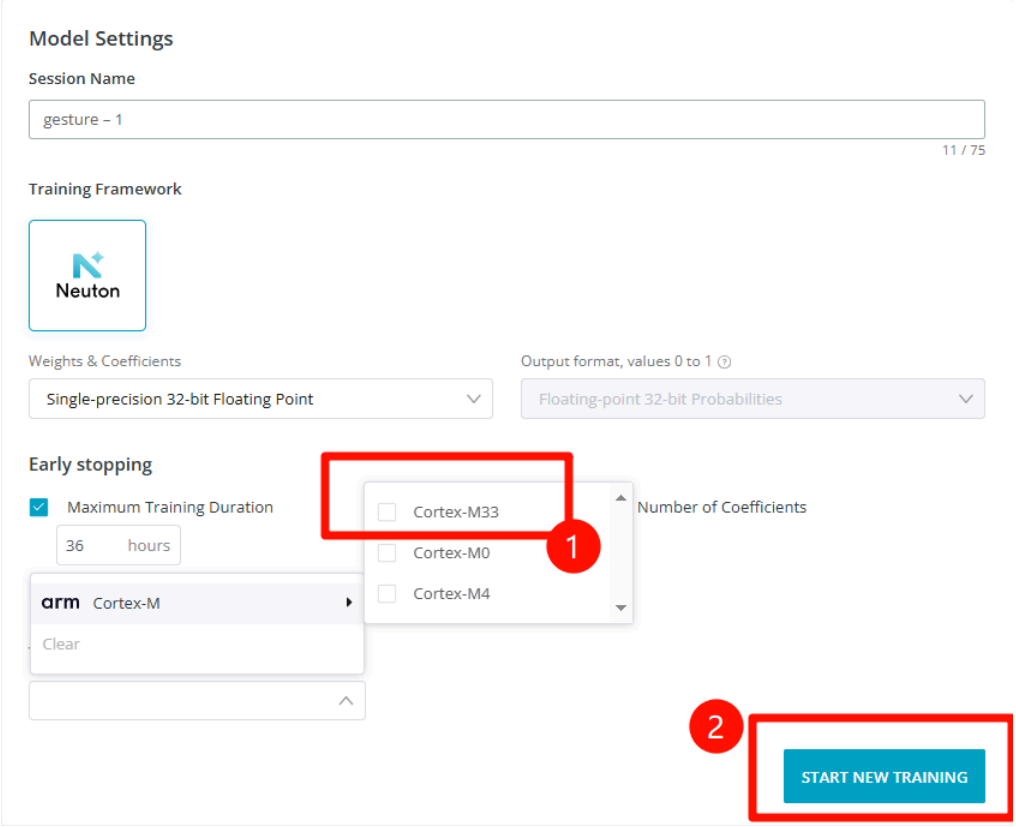


配置信号处理，窗口为行数，数值50，意思为这50行数据代表一次手势



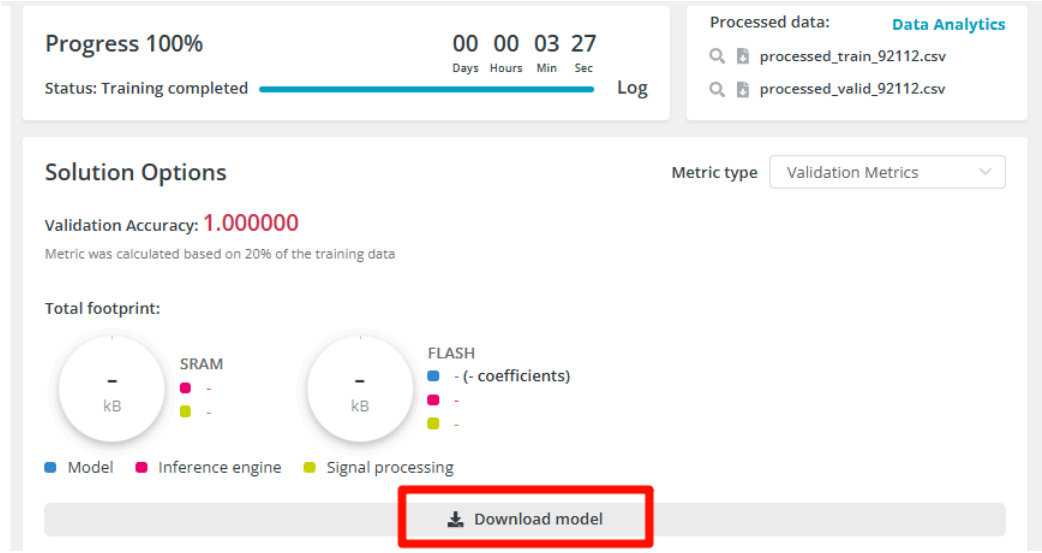
4. 模型训练

选择对应开发板的arm，54L15为M33，点击 **Start New Training** ，平台自动完成模型训练和验证：



5. 下载模型

训练完成后，下载模型压缩包



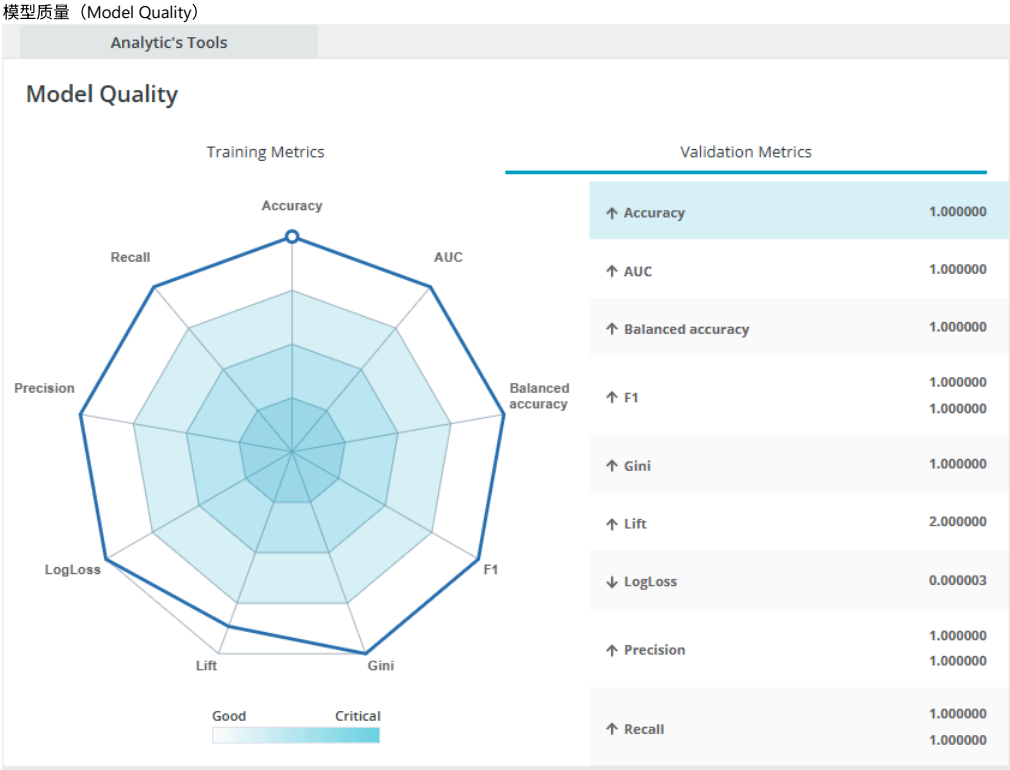
验证集指标表现极端完美：Accuracy、AUC、Balanced accuracy、F1、Gini、Precision、Recall 均为 1.000000，Lift 为 2.000000，LogLoss 仅为 0.000003，几乎为 0。（因为这是简单的二分类问题，而且两个手势差别较大）

内含**Neuton 模型： 编译好的 C 库，可直接嵌入 MCU 固件**

6. 分析模型

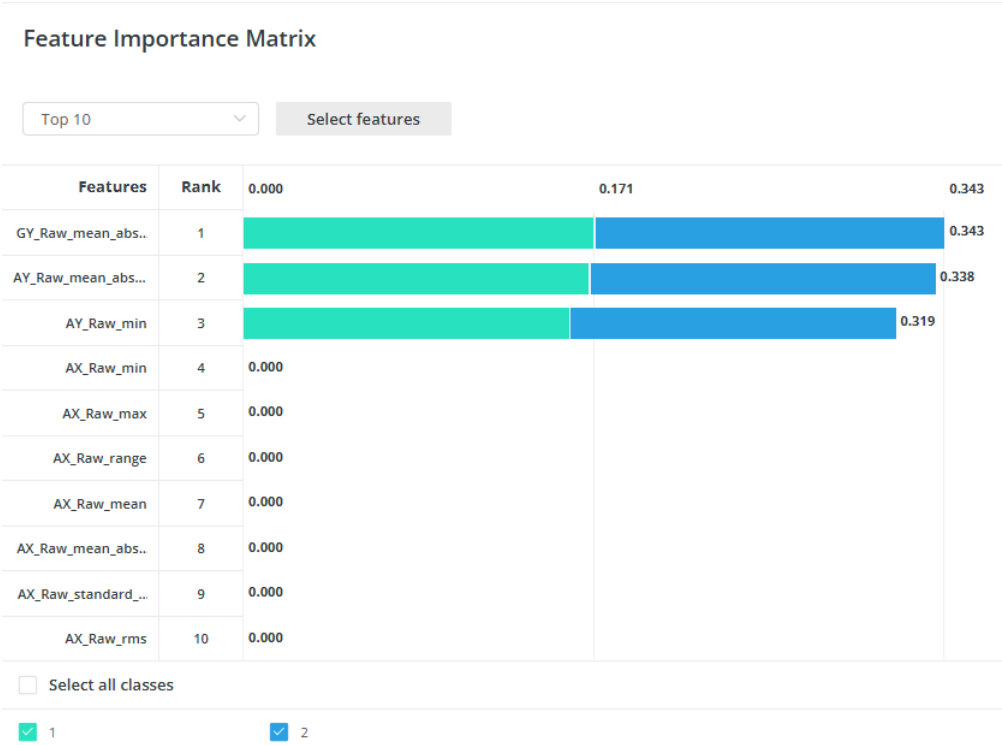
训练完成之后，eage ai也给出了一些分析图

1. 模型质量（Model Quality）



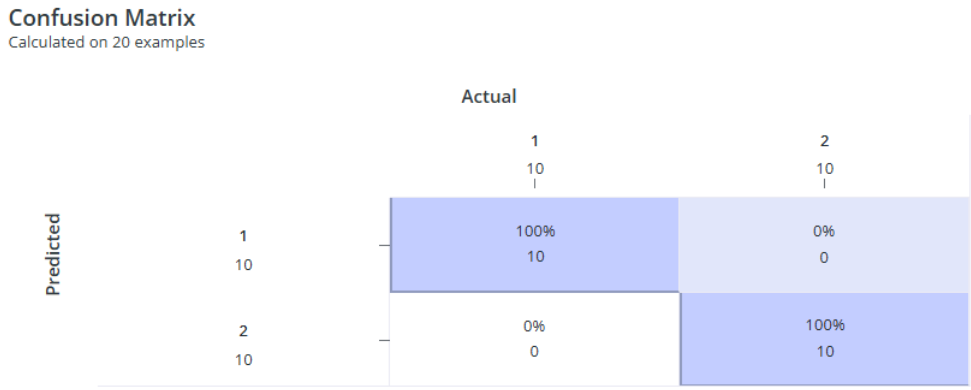
雷达图（训练集）：所有指标都接近 “Critical”（最优）区间，说明模型在训练集上也完全拟合。
潜在风险：这种 “满分” 表现极大概率是过拟合或数据泄露，尤其验证集样本量很小（混淆矩阵仅 20 个样本），模型泛化能力存疑（这很正常，因为这只是我一个人提供的少量数据集）

2. 混淆矩阵（Confusion Matrix）



验证集仅 20 个样本，类别 1 和类别 2 各 10 个。
预测结果：类别 1 全对（10/10），类别 2 全对（10/10），100% 准确率。
问题：样本量过少，无法代表真实场景，这个完美结果不具备统计可靠性。

3. 特征重要性（Feature Importance Matrix）



前 3 个特征贡献了绝大部分预测能力：
GY_Raw_mean_abs...：0.343
AY_Raw_mean_abs...：0.338
AY_Raw_min：0.319
其余 7 个特征重要性为 0，说明模型高度依赖这 3 个加速度相关的统计特征。
特征高度集中，可能存在特征冗余或单一特征主导预测的问题。

5.模型应用

基于例程workspace\edge-ai\samples\nrf_edgeai\classification

1. nrf_edgeai_user_model.c

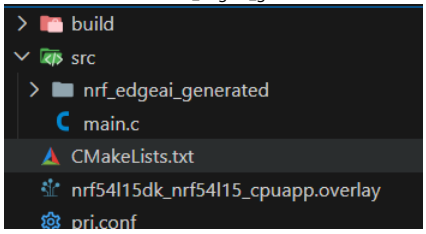
先看一下刚刚下载好的模型文件nrf_edgeai_user_model.c

```
////////////////////////////////////////////////////////////////////////////////
// 🔥 模型基本信息
#define EDGEAI_LAB_SOLUTION_ID_STR      "92168"   // 模型ID
#define EDGEAI_RUNTIME_VERSION_COMBINED 0x00000202 // 运行库版本

////////////////////////////////////////////////////////////////////////////////
// 🔥 输入配置（和BMI270强相关！必须看）
#define INPUT_TYPE                        f32
#define INPUT_FEATURE_DATA_TYPE           NRF_EDGEAI_INPUT_F32

#define INPUT_UNIQ_FEATURES_NUM           6    // 输入6轴：AX,AY,AZ,GX,GY,GZ
#define INPUT_UNIQ_FEATURES_USED_NUM      3    // 模型只用前3轴（加速度）
#define INPUT_WINDOW_SIZE                 50   // 推理需要50帧数据
#define INPUT_WINDOW_SHIFT                50   // 每50帧推理一次
#define INPUT_SUBWINDOW_NUM                0

// 🔥 输入归一化范围（传感器量程必须匹配：加速度±2G，陀螺仪±500dps）
static const nrf_user_input_t INPUT_FEATURES_SCALE_MIN[] = {
 -4.6521502, -8.0429897, -8.3255596, -11.8869753, -13.5773335, -4.6227450 };
static const nrf_user_input_t INPUT_FEATURES_SCALE_MAX[] = {
 2.4222181, 3.5064240, 2.8014641, 10.3522453, 11.5793114, 19.6133003 };
```

看这个文件的顶部定义可以得到以下信息

- 输入 6 轴、只用 3 轴加速度
- 50 帧一推理
- 传感器量程必须匹配
- 二分类输出 0/1
- DSP 提取 MCR/ZCR 特征

将下载好的模型文件夹nrf_edgeai_generated整个放进src目录下，替换掉原来的模型



2. main.c

通过刚才分析的信息配置对应的main.c
需要修改的地方

```
/* ===== Initialize Edge AI Model ===== */
model = nrf_edgeai_user_model_92168();
//修改为对应的模型函数
```

```
/* Model Configuration — must match generated model */
static const size_t USER_UNIQ_INPUTS_NUM  = 6;  /* AX, AY, AZ, GX, GY, GZ */
static const size_t USER_MODELS_CLASS_NUM = 2;  /* 2 classification classes */
// 修改为对应的输入和使用的输入
```

然后配置BIM270的设备树，把读取数值的程序加进去
以及加入静止检测，比如传感器值的变化小于多少时显示静止
完整的main.c

```
/*
 * SPDX-License-Identifier: LicenseRef-Nordic-5-Clause
 */

#include <zephyr/kernel.h>
#include <zephyr/device.h>
#include <zephyr/drivers/sensor.h>
#include <nrf_edgeai/nrf_edgeai.h>
#include "nrf_edgeai_generated/nrf_edgeai_user_model.h"
#include <zephyr/logging/log.h>
#include <stdlib.h>
#include <math.h>

LOG_MODULE_REGISTER(sensor_inference, LOG_LEVEL_INF);

/* Model Configuration — must match generated model */
static const size_t USER_UNIQ_INPUTS_NUM  = 6;  /* AX, AY, AZ, GX, GY, GZ */
static const size_t USER_MODELS_CLASS_NUM = 2;  /* 2 classification classes */

/* Sensor configuration */
#define SAMPLING_PERIOD_MS 20  /* 50Hz sampling rate */
```

做对应动作可以看到对应输出

```
I: [-2.762,  0.128,  9.301,   0.03,  -0.15,   0.24] -> STATIC [gyro_change=0.000]
I: [-2.398,  8.154,  8.464,   0.35,  -0.92,  -1.57] -> Class 1 (94%)
I: [-2.500, -0.554, 10.297,   0.10,  -0.44,   1.19] -> Class 0 (98%)
I: [-2.454, -0.170,  9.119,   0.23,   0.03,   0.33] -> STATIC [gyro_change=0.000]
I: [ 5.511, -8.017,  5.558,  -1.65,   0.40,  -7.60] -> Class 0 (69%)
```

作者：杨于棋 创建时间：2026-03-25 19:29
最后编辑：杨于棋 更新时间：2026-03-26 11:09